WEEK 9 – MEDICAL SHOP MANAGEMENT SYSTEM

Sales and Customer Analytics Report

SCENARIO

The medical shop maintains information about medicines, suppliers, customers, sales, and stock details in a database. The management needs an efficient system to analyze sales performance, understand customer purchasing behavior, identify best-selling medicines, and monitor medicine stock. SQL queries are used to generate reports, calculate revenue, analyze customer purchases, and support effective business decisions.

1. Apply SQL Aggregate Functions

Aggregate functions perform calculations on multiple records and return a single summarized value.

Task 1: Calculate the total number of orders placed.

```
SQL> SELECT COUNT(*) AS TOTAL_ORDERS
  2  FROM ORDERS;

TOTAL_ORDERS
------------
          10
```

Task 2: Calculate the total revenue generated from sales.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE
  2  FROM ORDERS;

TOTAL_REVENUE
-------------
        24950
```

Task 3: Calculate the average order value.

```
SQL> SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  2  FROM ORDERS;

AVERAGE_ORDER_VALUE
-------------------
               2495
```

Task 4: Find the highest and lowest order amount.

```
SQL> SELECT
  2      MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_AMOUNT,
  3      MIN(TOTAL_AMOUNT) AS LOWEST_ORDER_AMOUNT
  4  FROM ORDERS;

HIGHEST_ORDER_AMOUNT LOWEST_ORDER_AMOUNT
-------------------- -------------------
                4500                1200
```

Task 5: Calculate total sales for a specific period.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS PERIOD_TOTAL_SALES
  2  FROM ORDERS
  3  WHERE ORDER_DATE BETWEEN
  4        TO_DATE('01-10-2026', 'DD-MM-YYYY')
  5    AND TO_DATE('31-10-2026', 'DD-MM-YYYY');

PERIOD_TOTAL_SALES
------------------
```

2.Generate Total Sales Reports

Task 1: Calculate the total number of orders.

```
SQL> SELECT COUNT(ORDER_ID) AS TOTAL_ORDERS
  2  FROM ORDERS;

TOTAL_ORDERS
------------
          10
```

Task 2: Calculate total revenue generated from sales.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE
  2  FROM ORDERS;

TOTAL_REVENUE
-------------
        24950
```

Task 3: Calculate average order value.

```
SQL> SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  2  FROM ORDERS;

AVERAGE_ORDER_VALUE
-------------------
               2495
```

Task 4: Display the highest and lowest order amount.

```
SQL> SELECT
  2      MAX(TOTAL_AMOUNT) AS HIGHEST_AMOUNT,
  3      MIN(TOTAL_AMOUNT) AS LOWEST_AMOUNT
  4  FROM ORDERS;

HIGHEST_AMOUNT LOWEST_AMOUNT
-------------- -------------
          4500          1200
```

Task 5: Calculate total sales for a particular month.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS MONTHLY_SALES
  2  FROM ORDERS
  3  WHERE ORDER_DATE >= DATE '2026-10-01'
  4    AND ORDER_DATE < DATE '2026-11-01';

MONTHLY_SALES
-------------
```

3. Customer Purchase Analysis

Task 1: Calculate the total number of orders placed by each customer.

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      COUNT(ORDER_ID) AS TOTAL_ORDERS
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID;

CUSTOME TOTAL_ORDERS
------- ------------
C004               1
C009               1
C006               1
C007               1
C003               1
C005               1
C001               2
C002               2

8 rows selected.
```

Task 2: Calculate the total amount spent by each customer.

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID
  6  ORDER BY TOTAL_SPENDING DESC;

CUSTOME TOTAL_SPENDING
------- --------------
C001              5000
C007              4500
C002              4400
C004              3200
C005              2750
C009              2100
C003              1800
C006              1200

8 rows selected.
```

Task 3: Calculate the average spending per customer.

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      AVG(TOTAL_AMOUNT) AS AVERAGE_SPENDING
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID
  6  ORDER BY AVERAGE_SPENDING DESC;

CUSTOME AVERAGE_SPENDING
------- ----------------
C007                4500
C004                3200
C005                2750
C001                2500
C002                2200
C009                2100
C003                1800
C006                1200

8 rows selected.
```

Task 4: Identify customers with the maximum purchase amount.

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID
  6  HAVING SUM(TOTAL_AMOUNT) = (
  7      SELECT MAX(CUSTOMER_SPENDING)
  8      FROM (
  9          SELECT SUM(TOTAL_AMOUNT) AS CUSTOMER_SPENDING
 10          FROM ORDERS
 11          GROUP BY CUSTOMER_ID
 12      )
 13  );

CUSTOME TOTAL_SPENDING
------- --------------
C001              5000
```

Task 5: Identify customers with fewer purchases.

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      COUNT(ORDER_ID) AS TOTAL_PURCHASES
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID
  6  HAVING COUNT(ORDER_ID) = (
  7      SELECT MIN(PURCHASE_COUNT)
  8      FROM (
  9          SELECT COUNT(ORDER_ID) AS PURCHASE_COUNT
 10          FROM ORDERS
 11          GROUP BY CUSTOMER_ID
 12      )
 13  );

CUSTOME TOTAL_PURCHASES
------- ---------------
C004                  1
C009                  1
C006                  1
C007                  1
C003                  1
C005                  1

6 rows selected.
```

4. Find Top Customers Based on Purchase Amount

Task 1: Display the top five customers based on total spending.

```
SQL> SELECT *
  2  FROM (
  3      SELECT
  4          CUSTOMER_ID,
  5          SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  6      FROM ORDERS
  7      GROUP BY CUSTOMER_ID
  8      ORDER BY TOTAL_SPENDING DESC
  9  )
 10  WHERE ROWNUM <= 5;

CUSTOME TOTAL_SPENDING
------- --------------
C001              5000
C007              4500
C002              4400
C004              3200
C005              2750
```

Task 2: Identify customers with the maximum number of orders.

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      COUNT(ORDER_ID) AS NUMBER_OF_ORDERS
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID
  6  HAVING COUNT(ORDER_ID) = (
  7      SELECT MAX(ORDER_COUNT)
  8      FROM (
  9          SELECT COUNT(ORDER_ID) AS ORDER_COUNT
 10          FROM ORDERS
 11          GROUP BY CUSTOMER_ID
 12      )
 13  );

CUSTOME NUMBER_OF_ORDERS
------- ----------------
C001                   2
C002                   2
```

Task 3: Identify frequent customers. NHG

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      COUNT(ORDER_ID) AS NUMBER_OF_ORDERS
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID
  6  HAVING COUNT(ORDER_ID) >= 2
  7  ORDER BY NUMBER_OF_ORDERS DESC;

CUSTOME NUMBER_OF_ORDERS
------- ----------------
C002                   2
C001                   2
```

Task 4: Identify high-value customers.

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID
  6  HAVING SUM(TOTAL_AMOUNT) >= 10000
  7  ORDER BY TOTAL_SPENDING DESC;

no rows selected
```

5. Identify Best-Selling Medicines

Task 1: Identify medicines with the maximum sales quantity.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD
  4  FROM PRODUCT P
  5  JOIN ORDER_DETAILS OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY TOTAL_QUANTITY_SOLD DESC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P109
Metformin
                  5

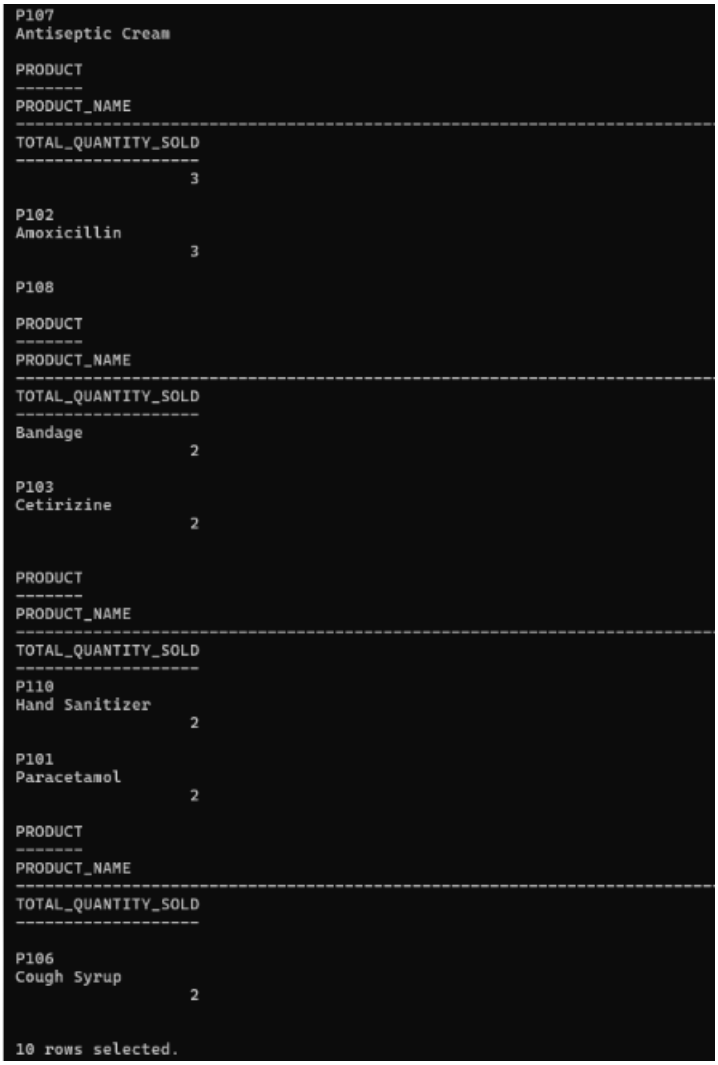
P105
Vitamin C
                  5

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

P104
Omeprazole
                  4

P107
Antiseptic Cream

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
```



Task 2: Identify medicines generating the highest revenue.



```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(OD.PRICE * OD.QUANTITY) AS TOTAL_REVENUE
  4  FROM PRODUCT P
  5  JOIN ORDER_DETAILS OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY TOTAL_REVENUE DESC;

PRODUCT
_______
PRODUCT_NAME
________________________________________________________________________________
TOTAL_REVENUE
_____________
P107
Antiseptic Cream
            4500

P109
Metformin
            2500

PRODUCT
_______
PRODUCT_NAME
________________________________________________________________________________
TOTAL_REVENUE
_____________

P104
Omeprazole
            2200

P105
Vitamin C

PRODUCT
_______
PRODUCT_NAME
________________________________________________________________________________
TOTAL_REVENUE
_____________
            2750

P102
Amoxicillin
         2499.99

P108

PRODUCT
_______
PRODUCT_NAME
________________________________________________________________________________
TOTAL_REVENUE
_____________
Bandage
            2100

P110
Hand Sanitizer
            1600


PRODUCT
_______
PRODUCT_NAME
________________________________________________________________________________
TOTAL_REVENUE
_____________
P103
Cetirizine
            1800

P101
Paracetamol
            1500

PRODUCT
_______
PRODUCT_NAME
________________________________________________________________________________
TOTAL_REVENUE
_____________

P106
Cough Syrup
            1200


10 rows selected.
```

Task 3: Identify least-selling medicines.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         NVL(SUM(OD.QUANTITY), 0) AS QUANTITY_SOLD
  4  FROM PRODUCT P
  5  LEFT JOIN ORDER_DETAILS OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY QUANTITY_SOLD ASC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------
P108
Bandage
            2

P101
Paracetamol
            2

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------

P110
Hand Sanitizer
            2

P103
Cetirizine

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------
            2

P106
Cough Syrup
            2

P107

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------
Antiseptic Cream
            3

P102
Amoxicillin
            3


PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------
P104
Omeprazole
            4

P105
Vitamin C
            5

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------

P109
Metformin
            5


10 rows selected.
```

Task 4: Identify medicines requiring promotion.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         NVL(SUM(OD.QUANTITY), 0) AS QUANTITY_SOLD
  4  FROM PRODUCT P
  5  LEFT JOIN ORDER_DETAILS OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY QUANTITY_SOLD ASC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------
P108
Bandage
            2

P101
Paracetamol
            2

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------

P110
Hand Sanitizer
            2

P103
Cetirizine

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------
            2

P106
Cough Syrup
            2

P107

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------
Antiseptic Cream
            3

P102
Amoxicillin
            3


PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------
P104
Omeprazole
            4

P105
Vitamin C
            5

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------

P109
Metformin
            5


10 rows selected.
```

6. Perform Category-wise Sales Analysis

Task 1: Calculate total sales generated by each medicine category.



```
SQL> SELECT C.CATEGORY_ID,
  2         C.CATEGORY_NAME,
  3         SUM(OD.PRICE * OD.QUANTITY) AS TOTAL_SALES
  4  FROM CATEGORY C
  5  JOIN PRODUCT P
  6  ON C.CATEGORY_ID = P.CATEGORY_ID
  7  JOIN ORDER_DETAILS OD
  8  ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 10  ORDER BY TOTAL_SALES DESC;

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
C007
Skin Care
       4500

C009
Diabetes Care
       3500

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------

C004
Digestive Care
       3200

C005
Vitamins

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
       2750

C002
Antibiotics
    2499.99

C008

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
First Aid
       2100

C010
Personal Care
       1900


CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
C003
Allergy Medicine
       1800

C001
Pain Relief
       1500

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------

C006
Cough and Cold
       1200


10 rows selected.
```

Task 2: Calculate the number of medicines sold in each category.



```
SQL> SELECT C.CATEGORY_ID,
  2         C.CATEGORY_NAME,
  3         NVL(SUM(OD.QUANTITY), 0) AS TOTAL_MEDICINES_SOLD
  4  FROM CATEGORY C
  5  LEFT JOIN PRODUCT P
  6  ON C.CATEGORY_ID = P.CATEGORY_ID
  7  LEFT JOIN ORDER_DETAILS OD
  8  ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 10  ORDER BY TOTAL_MEDICINES_SOLD DESC;

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD
--------------------
C005
Vitamins
                   5

C009
Diabetes Care
                   5

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD
--------------------

C004
Digestive Care
                   4

C002
Antibiotics

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD
--------------------
                   3

C007
Skin Care
                   3

C008

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD
--------------------
First Aid
                   2

C001
Pain Relief
                   2


CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD
--------------------
C006
Cough and Cold
                   2

C003
Allergy Medicine
                   2

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD
--------------------

C010
Personal Care
                   2


10 rows selected.
```

Task 3: Identify the highest revenue-generating category.



```
SQL> SELECT C.CATEGORY_ID,
  2         C.CATEGORY_NAME,
  3         SUM(OD.PRICE * OD.QUANTITY) AS TOTAL_REVENUE
  4  FROM CATEGORY C
  5  JOIN PRODUCT P
  6  ON C.CATEGORY_ID = P.CATEGORY_ID
  7  JOIN ORDER_DETAILS OD
  8  ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 10  ORDER BY TOTAL_REVENUE DESC;

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_REVENUE
-------------
C007
Skin Care
         4500

C009
Diabetes Care
         3500

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_REVENUE
-------------

C004
Digestive Care
         3200

C005
Vitamins

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_REVENUE
-------------
         2750

C002
Antibiotics
      2499.99

C008

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_REVENUE
-------------
First Aid
         2100

C010
Personal Care
         1900


CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_REVENUE
-------------
C003
Allergy Medicine
         1800

C001
Pain Relief
         1500

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_REVENUE
-------------

C006
Cough and Cold
         1200


10 rows selected.
```

Task 4: Calculate average sales per category.



```
SQL> SELECT C.CATEGORY_ID,
  2         C.CATEGORY_NAME,
  3         AVG(OD.PRICE * OD.QUANTITY) AS AVERAGE_SALES
  4  FROM CATEGORY C
  5  JOIN PRODUCT P
  6  ON C.CATEGORY_ID = P.CATEGORY_ID
  7  JOIN ORDER_DETAILS OD
  8  ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 10  ORDER BY AVERAGE_SALES DESC;

CATEGOR
------
CATEGORY_NAME
--------------------------------------------------------------------------------
AVERAGE_SALES
-------------
C007
Skin Care
         4500

C009
Diabetes Care
         3500

CATEGOR
------
CATEGORY_NAME
--------------------------------------------------------------------------------
AVERAGE_SALES
-------------

C004
Digestive Care
         3200

C005
Vitamins

CATEGOR
------
CATEGORY_NAME
--------------------------------------------------------------------------------
AVERAGE_SALES
-------------
         2750

C002
Antibiotics
      2499.99

C008

CATEGOR
------
CATEGORY_NAME
--------------------------------------------------------------------------------
AVERAGE_SALES
-------------
First Aid
         2100

C010
Personal Care
         1900


CATEGOR
------
CATEGORY_NAME
--------------------------------------------------------------------------------
AVERAGE_SALES
-------------
C003
Allergy Medicine
         1800

C001
Pain Relief
         1500

CATEGOR
------
CATEGORY_NAME
--------------------------------------------------------------------------------
AVERAGE_SALES
-------------

C006
Cough and Cold
         1200


10 rows selected.
```

7. Generate Business Analytics Reports

Report 1: Sales Performance Report

This report displays total sales, number of orders, average order value, and highest order value.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES,
  2         COUNT(ORDER_ID) AS NUMBER_OF_ORDERS,
  3         AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE,
  4         MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_VALUE
  5  FROM ORDERS;

TOTAL_SALES NUMBER_OF_ORDERS AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
----------- ---------------- ------------------- -------------------
      24950               10                2495                4500
```

Report 2: Customer Analytics Report

This report displays customer names, number of orders, total spending, and purchase frequency.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS,
  4         NVL(SUM(O.TOTAL_AMOUNT), 0) AS TOTAL_SPENDING
  5  FROM CUSTOMER C
  6  LEFT JOIN ORDERS O
  7  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  9  ORDER BY TOTAL_SPENDING DESC;

CUSTO CUSTOMER_NAME                  NUMBER_OF_ORDERS TOTAL_SPENDING
----- ------------------------------ ---------------- --------------
C001  Devi                                          2           5000
C007  Sindhu                                        1           4500
C002  Radhi                                         2           4400
C004  Vijay                                         1           3200
C005  Abi                                           1           2750
C009  Sanjay                                        1           2100
C003  Ajay                                          1           1800
C006  Anu                                           1           1200

8 rows selected.
```

Report 3: Medicine Performance Report

This report displays medicine names, quantities sold, and revenue generated.



Report 4: Category Analysis Report

This report displays category names, total medicines sold, and total revenue.



```
SQL> SELECT C.CATEGORY_ID,
  2         C.CATEGORY_NAME,
  3         NVL(SUM(OD.QUANTITY), 0) AS TOTAL_MEDICINES_SOLD,
  4         NVL(SUM(OD.PRICE * OD.QUANTITY), 0) AS TOTAL_REVENUE
  5  FROM CATEGORY C
  6  LEFT JOIN PRODUCT P
  7  ON C.CATEGORY_ID = P.CATEGORY_ID
  8  LEFT JOIN ORDER_DETAILS OD
  9  ON P.PRODUCT_ID = OD.PRODUCT_ID
 10  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 11  ORDER BY TOTAL_REVENUE DESC;

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD TOTAL_REVENUE
-------------------- -------------
C007
Skin Care
                   3          4500

C009
Diabetes Care
                   5          2500

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD TOTAL_REVENUE
-------------------- -------------

C004
Digestive Care
                   4          2200

C005
Vitamins

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD TOTAL_REVENUE
-------------------- -------------
                   5          2750

C002
Antibiotics
                   3       2499.99

C008

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD TOTAL_REVENUE
-------------------- -------------
First Aid
                   2          2100

C010
Personal Care
                   2          1000


CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD TOTAL_REVENUE
-------------------- -------------
C003
Allergy Medicine
                   2          1800

C001
Pain Relief
                   2          1500

CATEGOR
-------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_MEDICINES_SOLD TOTAL_REVENUE
-------------------- -------------

C006
Cough and Cold
                   2          1200


10 rows selected.
```

CONCLUSION

The Medical Shop Management System helps analyze sales performance, customer purchasing behavior, medicine demand, and category-wise revenue. SQL aggregate functions and joins help generate useful business reports. These reports support better stock management, identify popular medicines, and help the shop make informed business decisions.

Note: The table and column names in this draft are examples. Confirm them against your actual database before running the queries.